Cornell University
College of Arts & Sciences - Department of Statistical Science

STSCI 5720 - Applied Neural Networks

A Multi-Model Neural Network Approach to Classifying AI-Generated and Human-Written Essays

Yik Hau Au | Ariel Sheynzon | Mengqi Shi | Candy Sizhe Wu | Sharon Zou

Data Exploration / Preprocessing

Today, deciphering whether content is written by a human or artificially generated has become extremely difficult, especially with the rise of extremely powerful AI models such as ChatGPT, Gemini, and others. So, perhaps counterintuitively, our group decided to test whether AI models like RNNs, LSTMs, and others could determine what is human and what is not.

To aid us in our research, we found the following dataset on Kaggle to analyze: AI(LLMS) vs. Human Texts Cleaned and Optimized. This dataset was compiled from 6 different sources and cleaned for classification and NLP tasks. In total, there are 1,847,783 rows, such that each row represents an essay that is either classified as human-written text or as AI-generated. There are only 2 columns in the dataset: label and clean_text. Label is a binary label indicating the source of the text (0 for human or 1 for AI), and clean_text is the preprocessed and cleaned text content.

The dataset was preprocessed in the following ways. (1) Unwanted characters, extra spaces, and special symbols (such as uppercase letters and punctuation) have been removed from all the text entries. (2) Each entry is labelled with a 0 or a 1 to ensure label consistency. (3) Unnecessary columns have been removed, and data types have been downcast to more efficient forms for memory optimization. (4) Duplicate rows have been removed. (5) Missing or null values have been handled to maintain data quality. (6) The dataset only includes two columns: label and clean_text, ensuring a compact size.

In addition to the preprocessing completed on Kaggle, we also (1) trained our models on a smaller size of 30k rather than the almost 2 million rows, (2) checked that our subset contained a roughly 50/50 split of the two labels (16065 human written and 13935 AI-generated observations), and (3) set a seed of 123 to ensure that we can efficiently run our models, confirm that our small sample size is not biased, and ensure reproducible results.

Before we could train the models, we first converted the essay text into a numerical format that the neural network could process. Initially, we tried character-to-integer encoding. However, this approach was inefficient, as it produced very long sequences, which meant increased computational cost. Next, we tried Word2Vec embeddings, which convert words to vector representations, capturing semantic information, but training it on this large dataset was again too slow. Finally, we discovered PyTorch's nn.Embedding, which converts words to numerical vectors during training. We chose to utilize this method as it is much faster than the previous two methods. The first step involved tokenization, which splits each essay into individual words. Then, we built the vocabulary by collecting all unique words that appeared in the essays, along with two special tokens: padding, which is used to make sequences the same length when training in batches, and unknown words, which are used for words that do not appear in the vocabulary. Each essay was converted into a sequence of integers, where each number represented a word. Then, because essays have different lengths, sequences were padded using the special token we created earlier so that all inputs in the same batch have the same size to prevent errors. Finally, the dataset was split 80/20 into training and testing sets, where 80% of the data was used for training, and 20% were used for evaluation. We utilized Data Loaders to batch the essays to feed into the model during training.

We trained our data on six different neural networks (Single-Layer RNN, Multi-Layer RNN, LSTM, RNN Auto-Encoder, LSTM Auto-Encoder, and Transformer) to see which model would be able to classify the text the best.

1. Single-Layer RNN

Because we are analyzing text data, which is sequential, we began with recurrent neural networks (RNNs), commonly used as the first neural network architecture for many natural language processing (NLP) tasks. RNNs serve as a simple baseline to capture sequence structure (i.e. order, context, relationship), and even a single layer can learn useful sequential patterns like negation or stylistic cues in text. By beginning with this model, we established performance benchmarks for comparing and evaluating the improvements of more complex models.

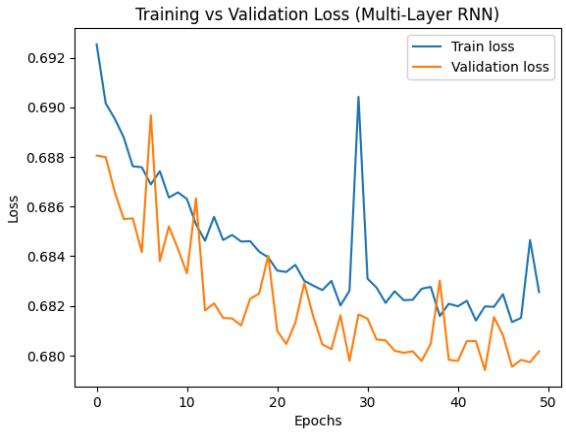
We selected a larger hidden state size of 128 to help retain more context, as these essay texts are longer sequences. Initially, we processed each essay in a batch one at a time, and with 24,000 essays across 50 epochs, this resulted in approximately 1.2 million forward passes. Moreover, the computation was fully sequential, leading to very slow training. Consequently, we passed in the train and test sets from DataLoader, which automatically grouped the dataset into batches of 64 essays, which were processed in one forward pass, which resulted in much faster computation of 50 epochs in 15 minutes. Using the Adam optimizer with a learning rate of 0.001 and negative log likelihood loss, we observed fluctuations in both training and validation losses that remain overall flat, with a sharp decline around epoch 35 followed by another steep increase 10 epochs later (Figure 1.1), reaching a final accuracy of 53.44% for the train set and 54.20% for the test set (Table 1) after 50 epochs. While the single-layer RNN captured some signal in this dataset, it did not perform significantly better than random guessing for a binary classification task, suggesting that the model may not be expressive enough. So we next explored multi-layer RNNs, capable of learning richer and more hierarchical representations of the data.

2. Multi-Layer RNN

Multi-layer RNNs learn more hierarchical language features, like word usage, syntax, and stylistic cues, and integrate contextual information from earlier layers, allowing the model to learn more complex patterns that can help distinguish between AI-generated and human-written texts.

Keeping the same batch size and hidden state size, we used 3 RNN layers. Because of this deeper architecture, we added dropout = 0.2 as a regularization technique to zero out a fraction of outputs from neurons across the network, thereby controlling the flow of gradients through the network and combating the concerns of vanishing gradients in the shallow layers. We also lowered the learning rate to 0.0001 to allow stabilized training and ensure more gradual parameter updates. Across 50 epochs, we saw a much steadier decline with smaller fluctuations in both the training and validation losses (Figure 1.2); however, the model achieved approximately the same train (53.42%) and test (54.27%) accuracies (Table 1). One hypothesis we had for this outcome is that this dataset's human-written essays contain many typos, which is a shallower stylistic feature that can act as a reasonable indicator of human writing, so the additional layers could instead be learning noise, resulting in little improvement in classification accuracy. And even with deeper models, our standard RNNs still struggled with long-range textual dependencies. Thus, we

next explored Long Short-Term Memory (LSTM) networks, which incorporate a series of gates that allow the model to better retain information.



3. LSTM

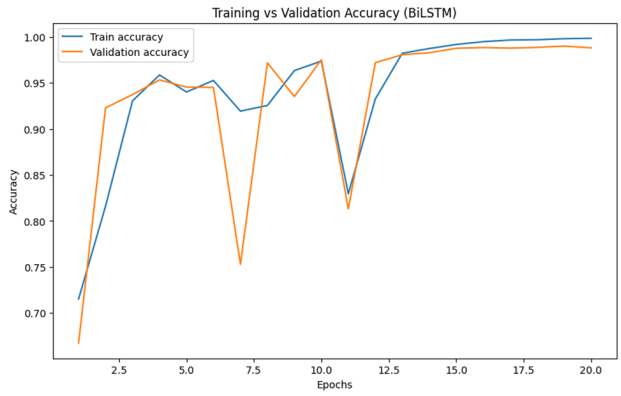
As we just discussed, RNNs usually struggle with long-term sequences because of their inability to capture longer textual dependencies. Since we are working with essays that can typically range anywhere from 300 to 800 words, this posed a challenge for the RNN. Thus, we needed to choose an architecture that can maintain information in memory for longer periods of time. Logically, we chose to explore an LSTM architecture.

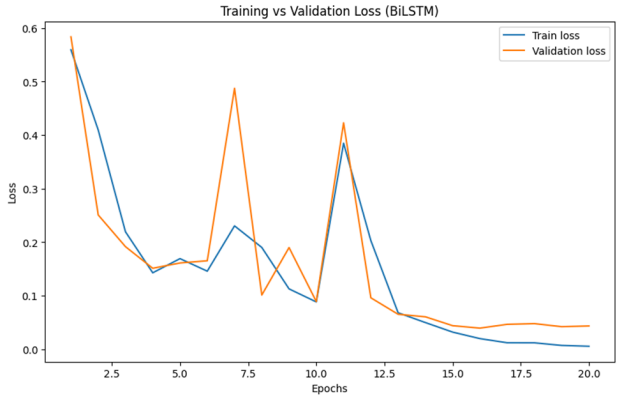
In the model, each word token was first converted into a dense vector using an embedding layer. These embeddings were then passed into a bidirectional LSTM layer. Using a bidirectional structure allowed the network to process the essay both forward and backward, incorporating context from words that appear both before and after a given token. The hidden representations produced by the LSTM were then used to create a single representation of the essay by selecting the final valid timestep corresponding to the last non-padding token in the sequence. This step is important because essays were padded to equal length during batching, and we wanted the model to focus only on the actual text. After this representation was obtained, dropout was applied again to reduce overfitting, and a fully connected layer produced the final classification output for whether the essay was written by a human or generated by AI.

The final part of the code implemented the training loop for the model. During training, the essays were passed through the network, the cross-entropy loss was calculated, and the model parameters were updated using the Adam optimizer. Accuracy and loss were tracked across epochs for both the training and validation sets to monitor how well the model is learning. This setup allowed us to evaluate how effectively the LSTM architecture captures patterns in the essays compared to the earlier RNN model.

The BiLSTM model showed strong performance over the 20 training epochs. Both training and validation loss decreased substantially after the first few epochs, indicating that the model was successfully learning patterns that differentiate human-written and AI-generated essays. Training accuracy improved from approximately 71% in the first epoch to over 99% by the final epoch, while validation accuracy also increased rapidly and stabilized around 98-99%. Although there were a few temporary spikes in both

training and validation loss around epochs 7 and 11, the model quickly recovered and continued improving, suggesting that the optimizer was still effectively converging. By the later epochs, both loss curves flattened and remained low, indicating that the model had largely converged by the twentieth epoch. Overall, the BiLSTM architecture demonstrated strong classification ability on the dataset, achieving high validation accuracy while maintaining relatively low validation loss. Below is a graph comparing training vs validation loss for the BiLSTM.





Although the LSTM improved the model's ability to handle longer text sequences, we were also interested in exploring whether learning deeper latent representations of essays could further improve performance. This motivated the next architecture we tested: RNN and LSTM auto-encoders.

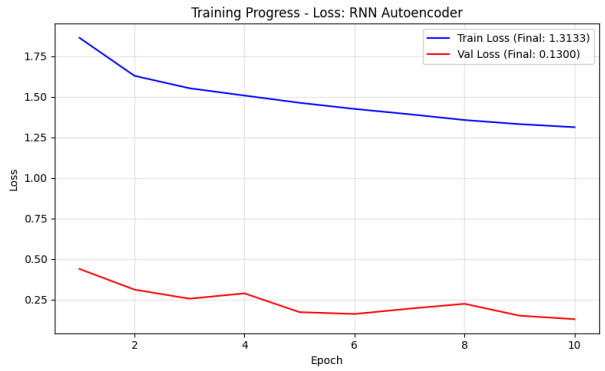
4. Auto-Encoders (RNN/LSTM)

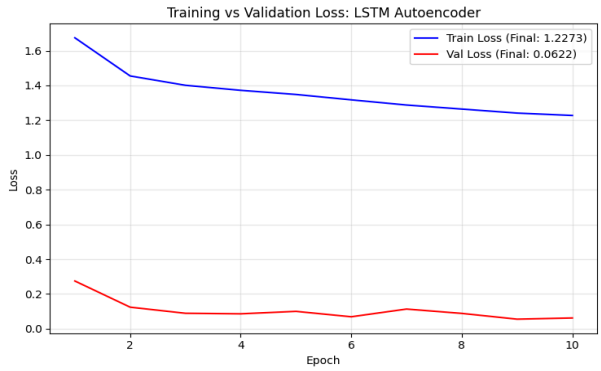
We now move on to the auto-encoder model. The assumption is that the model has a learning phase and an exam phase to learn the difference between AI and human text, namely the encoder and decoder phases. For the encoder phase, the embedding layer converts word indices to dense vectors (dimension = 32). The model (RNN or LSTM) then processes the sequence and produces hidden states. Finally, the final hidden state captures the entire sequence representation before sending it to the bottleneck. The bottleneck compresses the sequence into a dense "block" (hidden dimension = 64) that contains only the most meaningful features. For the decoder, we reconstruct the input sequence token by token from this bottleneck representation. This architecture forces the model to learn a "compressed" understanding of writing styles—such as the natural "typos" and irregularities in human writing—by requiring it to mimic the input features perfectly during reconstruction.

This upgrades the simple LSTM/RNN model by introducing a multi-task learning strategy. While a standard LSTM only optimizes for classification, the auto-encoder adds a reconstruction loss. This forces the model to prioritize structural and stylistic details (like common misspellings) that might be ignored by a pure classifier but are critical for identifying human-written text. By learning to "rebuild" the essay, the encoder develops a much richer feature set in the bottleneck, which the linear classifier then uses to achieve higher accuracy.

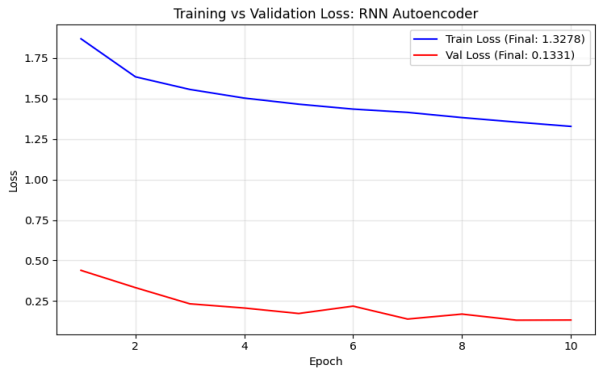
However, training auto-encoders introduces significant computational challenges. We observed that the reconstruction task significantly increases RAM usage because the model must store gradients for every

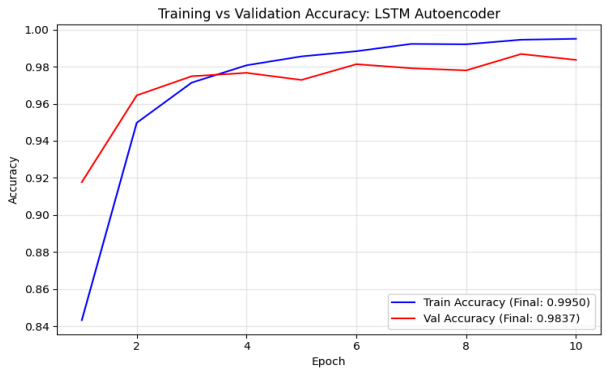
token in the output sequence. To prevent the CUDA GPU terminal from crashing, we had to lower the batch size (e.g., to 32 for LSTM) and impose a truncation on the input data to the first 150 tokens, as sizes starting at 200 would exceed memory limits. We also added gradient clipping (max norm = 1.0) to stabilize training against exploding gradients in long sequences.





The results validated these choices. The RNN auto-encoder converged promptly, reaching approximately 97% final accuracy, which is significantly higher than the accuracy of standard RNN. The LSTM auto-encoder yielded even better results, reaching 94% validation accuracy in the first epoch and climbing to 98.3% after 10 epochs. To further investigate methods to improve this, a Transformer model is explored next. Unlike the sequential nature of RNNs/LSTMs, Transformers use self-attention mechanisms to process the entire sequence in parallel, allowing the model to capture long-range dependencies and subtle stylistic nuances more effectively.





5. Transformer

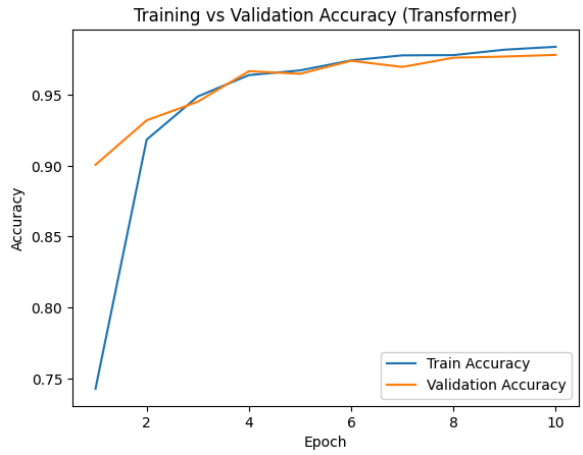
Transformer methods rely on multi-head attention mechanisms to capture relationships between words in a sequence, enabling the model to learn contextual dependencies more effectively than traditional sequential models such as RNN or LSTM. Additionally, transformers give more efficient computation with less running time, which can be seen in Table 1.

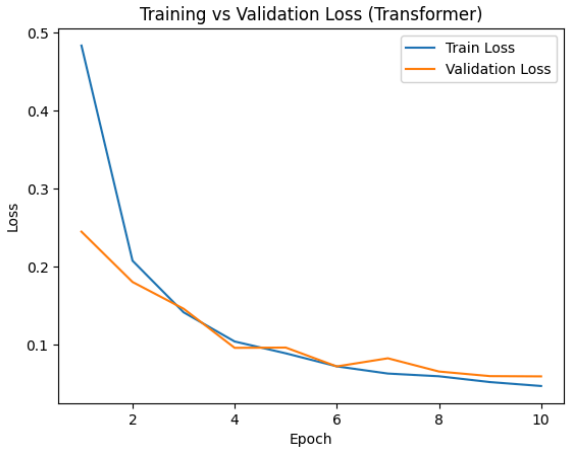
Our model begins with a word embedding layer that converts each token in the input text into a vector representation. Since transformers do not inherently encode sequence order, a positional encoding module is added to incorporate information about the position of each token in the sequence. The transformer

encoder is composed of multiple layers containing multi-head attention and feedforward neural networks, which allow the model to capture both local and long-range dependencies in the text. After the sequence passes through the encoder layers, the representation according to the classification token (CLS token) serves as a summary of the entire input sequence and is used for classification. Finally, a fully connected linear layer maps the CLS representation to two output classes: human-written or AI-generated.

We also carefully chose the parameters to balance the model performance and computational efficiency. The transformer encoder uses 8 attention heads, allowing the model to attend to multiple contextual relationships within the text simultaneously. The feedforward hidden dimension of 512 is larger than the embedding dimension of 256, and this enables more expressive nonlinear transformations after attention layers. The model contains 2 transformer layers such that it provides enough depths to learn meaningful patterns without significantly increasing training time. To reduce overfitting, we apply a dropout rate of 0.1 and weight decay of 1e-4 with the usage of Adam optimizer.

The transformer model was trained for 10 epochs, and the training process showed steady improvements in both loss and accuracy across epochs. At the beginning of training, the model had a training accuracy of 72.65% and validation accuracy of 88.68%. As the training progressed, both accuracies improved significantly as shown below. By the final epoch, our model achieved 98.49% training accuracy and 98.33% validation accuracy, which indicate the transformer model successfully learned meaningful patterns without severe overfitting. Interestingly, during early training epochs, the validation accuracy was higher than the training accuracy, and this phenomenon can occur due to dropout regularization which is active during training but disabled during evaluation. The training and validation accuracies became very close in the end, meaning that the model generalizes well to unseen data, and the high validation accuracy of 98.33% suggests the transformer architecture is effective for our classification task.





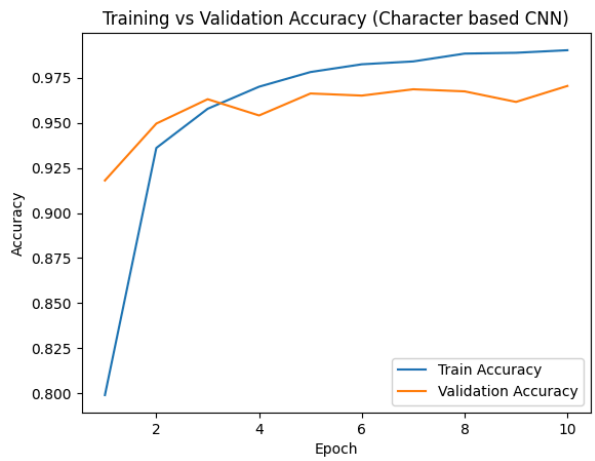
6. Area for Improvement

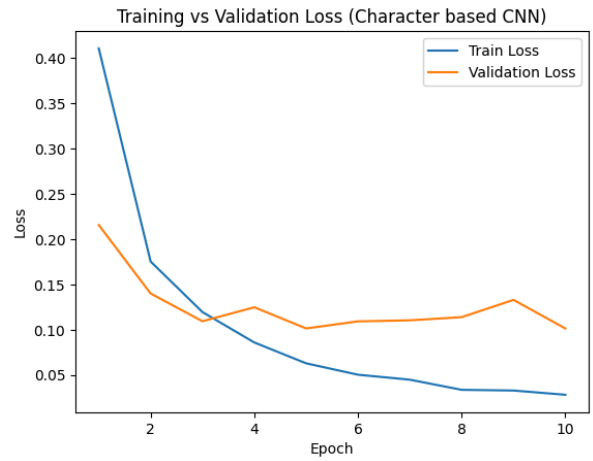
1. Character-based CNN

We also pay special attention to the character-based CNN, since character-level models may be useful in this task because they capture fine-grained textual information that may not be visible at the word level.

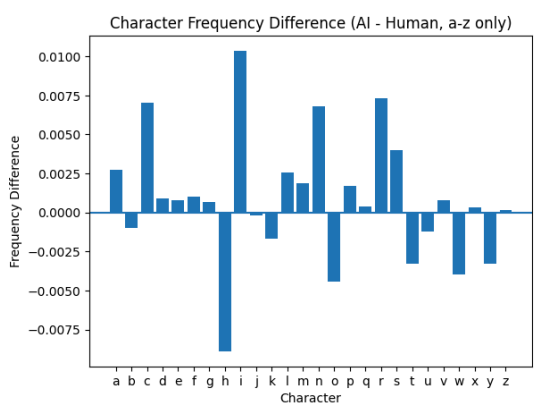
By directly operating on characters rather than words, the model can detect the subtle stylistic differences and remain robust to spelling variations or typos that may appear in the dataset.

Interestingly, the accuracy from character-based CNN was surprisingly good (more than 97.5%), and the runtime was much lower (2 min for 10 epoch). Since CNN is powerful in capturing local patterns, we completed research to explore if the dataset contains local features that strongly correlate with the label, making it particularly easier for CNN to classify.





For local feature detection, we first researched the character frequency discrepancy, which is the character frequency in AI-generated essays minus that of human-written. The left plot below shows that letter 'h, k, o, t, w, y' appear more frequently in human-written contents, while letter 'a, c, i, n, r, s' have higher frequencies in AI-generated essays. Moreover, we checked if certain bigrams, such as 'th', 'er', appear more in one class, and there are detectable differences in the frequencies and bigram types as in the table below. These local patterns may help explain why character-based CNN performed unexpectedly well.



Bigram	Human_Freq	AI_Freq	Diff(AI-Human)	Abs_Diff
ti	0.006306	0.010836	0.004529	0.004529
es	0.007934	0.011492	0.003557	0.003557
ha	0.008439	0.005806	-0.002633	0.002633
w	0.011219	0.007634	-0.003585	0.003585
ou	0.009887	0.006122	-0.003765	0.003765
he	0.018462	0.014296	-0.004166	0.004166
th	0.024752	0.019234	-0.005518	0.005518
t	0.031826	0.024962	-0.006864	0.006864
t	0.020456	0.013275	-0.007181	0.007181
e	0.039958	0.032753	-0.007205	0.007205

2. Vectorize errors comparison

Another potential area of improvement for our models would be to perform a deeper analysis of the essays that are consistently misclassified across the different architectures tested in this project. After identifying these incorrectly labeled samples, the essays could be converted into vector representations

and compared using cosine similarity, which measures the angle between vectors in the embedding space. Essays with a small angle between their vectors would indicate strong similarity, while larger angles would suggest that the misclassified samples are more heterogeneous. This analysis could help determine whether the models are consistently struggling with a particular type of essay, such as those with similar stylistic patterns, vocabulary usage, or sentence structure. If clusters of similar essays are repeatedly misclassified, this could indicate specific weaknesses in how the models capture certain linguistic features. Prior research has used embedding-space similarity measures, such as cosine similarity, to analyze classification errors and examine whether misclassified examples occupy overlapping regions in representation space (Xie et al.). Applying a similar approach here could provide insight into why the models fail on certain essays and help guide future improvements to the architectures or preprocessing pipeline.

3. Syntactic patterns of AI-written text

Moving on to more general written text, typos alone may not be sufficient to identify AI-written text. Instead, we can examine the structural features. One feature to investigate is sequential perplexity, which measures how “surprising” each next word is (Lindberg and Nobel). LLMs generate text sequentially by selecting the next token based on estimated probabilities. This is an auto-regressive process that depends on the preceding text, and it can yield more uniform predictability (less surprising) than human writing.

Lexical diversity is also a popular direction for feature engineering. Studies have shown that LLMs such as ChatGPT may exhibit lower lexical diversity than humans in writing (Reviriego et al.). This motivates constructing indicators based on lexical diversity measures for AI-written text identification, such as moving-average lexical diversity, type–token ratio variants, and frequency distributions of selected words.

Conclusion

After training the data on six different neural network models, we found that the Single-Layer RNN and Multi-Layer RNN performed the worst in terms of training and testing accuracy but were the best in terms of computational cost, as they had the fastest run time. Their accuracy scores were around 53-54%, which is only slightly better than randomly guessing, with a run time of 0.3-0.6 minutes per epoch. We also discovered that RNN Auto-Encoder, LSTM Auto-Encoder, and Transformer performed very well, with training and testing accuracies of 97-99%. But they unfortunately had very long run times, ranging from 4-6 minutes per epoch, due to the complexity of these models. Ultimately, we see that LSTM performed the best overall in terms of both accuracy (99.85% train accuracy and 98.82% test accuracy), and run time (1.25 minutes per epoch). LSTM's ability to learn and remember information in long sequences and discard irrelevant information, while being a comparatively simpler model, makes it perform well for our essay classification dataset.

Appendix

Table 1: Model performance metrics

Model	Train Accuracy	Test Accuracy	Run time (on T4 GPU)
Single-Layer RNN	53.44%	54.20%	15 min for 50 epochs (0.3 min per epoch)
Multi-Layer RNN	53.42%	54.27%	30 min for 50 epochs (0.6 min per epoch)
LSTM	99.85%	98.82%	25 min for 20 epochs (1.25 min per epoch)
RNN Auto-Encoder	97.63%	96.63%	1 hour for 10 epochs (6 min per epoch)
LSTM Auto-Encoder	99.54%	98.30%	1 hour for 10 epochs (6 min per epoch)
Transformer	98.49%	98.33%	40 min for 10 epochs (4 min per epoch)

Reference

1. Lindberg, Karl, and Isak Nobel. “Ai or Human? Detecting Machine-Generated Text through Perplexity.” *DIVA*, 8 July 2025, kth.diva-portal.org/smash/record.jsf?pid=diva2%3A1982607&dswid=1992 .
2. Reviriego, Pedro, et al. “Playing with Words: Comparing the Vocabulary and Lexical Diversity of ChatGPT and Humans.” *arXiv*, 14 Mar. 2025, `https://arxiv.org/abs/2308.07462`.
3. Xie, Xiaohui, et al. "Agentic Information Retrieval." arXiv, 2024. https://doi.org/10.48550/arXiv.2410.10153